Bangladesh University of Business and Technology



ASSIGNMENT

Assignment No : 01
Course Code : CSE 215
Course Title : Computer Architecture

Submitted By:

Name : Md. Junayet Hasan
ID No : 20255203046
Intake : 48
Section :
Program : B.Sc. Engg in CSE (DH)

Submitted To:

Name : Fahim Morshed

(Lecturer)
Dept. of CSE
Bangladesh University of Business & Technology

Date of Submission : 27/10/2026

1) Given,

```
int y = 15;
int count = 0;
int i = 0;
while (i<8){
    if (B[i]>y){
        count++;
    }
    i++;
}
```

let, y = $s1 = 15

count = $s2 = 0

i = $s3 = 0

B[0] Address Stored in = $s0

Converted Code:

```
.data
B: .word 10, 20, 5, 18, 30, 12, 16, 8
.text
.globl main
main:
    la  $s0, B
    li  $s1, 15
    li  $s2, 0
    li  s3, 0
Loop:
    slti $t0, $s3, 8
    beq  $t0, $zero, End
    sll $t1, $s3, 2
    add $t1, $s0, $t1
    lw  $t2, 0($t1)
    slt  $t3, $s1, $t2
    beq  $t3, $zero, skipcount
    addi $s2, $s2, 1
skipCount:
    addi  $s3, $s3, 1
    ~~syscall~~
    j  Loop

End:
    move  $a0, $s2
    li   $v0, 1
    Syscall
    li $v0, 10
    Syscall
```

2) Given,

```
int sum = 0;
int K = 0;
while (K<12) {
    sum+= x[K];
    K++;
}
```

$t0 = sum = Accumulator

$t1 = K = 0 = loop index

$s5 = Base Address of x, *x[0]

Converted Code:

```
.data
X: .word 1,2,3,4,5,6,7,8,9,10,11,12
.text
.globl main
main:
    la  $s5, X
    li  $t0, 0
    li  $t1, 0
Loop:
    slti  $t2, $t1, 12
    beq  $t2, $zero, End

    sll  $t3, $t1, 2
    add  $t3, $s5, $t3, $t3
    lw  $t4, 0($t3)
    add  $t0, $t0, $t4
    li $v0, 1 # int Print service = 1
    syscall
    addi  $t1, $t1, 1
    j  Loop
End:
    move  $a0, $t0
    li  $v0, 1  # 1 Int Print
    syscall

    li  $v0, 10  # 10 Exit serv
    syscall
```

3) Given

```
int limit = 20;
int total = 0;
int j = 0;
while (j < 10){
    if (D[j] <= limit){
        total += D[j];
    }
j++;
}
```

ut,

total = $V0

limit = $V1 = 20

j = $S4 → Array index

* D[0] = $S2 = Base Address of D

Converted Code:

```
.data
D: .word 10, 15, 25, 30, 5, 20, 22, 8, 40, 12
.text
.globl main
main:
    la $s2, D
    li $v0, 0
    li $v1, 20
    li $s4, 0
Loop:
    slti $t0, $s4, 10
    beq, $t0, $zero, End
    sll $t1, $s4, 2
    add $t1, $s4, $t1
    lw $t2, 0($t1)
    slt $t3, $v1, $t2
    bne $t3, $zero, SkipAdd
    add $v0, $v0, $t2
SkipAdd:
    addi $s4, s4, 1
    j Loop
End:
    move $a0, $v0
    li $v0, 1
    Syscall
    li $v0, 10
    Syscall
```

4) Convert Machine instruction (Assembly) from Hexadecimal Code:

Given,

0x01854020, 0x02264824, 0x01476822

Hexadecimal code	Hex to Binary	OPCode	rs	rt	rd	Shamt	funct
0x01854020	op → 00000 rs → 01100 rt → 00101 rd → 01000 shamt → 00000 funct → 100000	000000 " 0	01100 " 12 " $t4	00101 " 5 " $a1	01000 " 8 " $t0	00000 " 0	100000 " 32 " Add
0x2264824	000000 10001 00110 01001 00000 100100	000000 " 0	10001 " 17 " $s1	00110 " $6 $a2	01001 " 9 " $t1	00000 " 0	100100 " 36 " and
0x01476822	000000 01010 00111 01101 00000 100010	000000 " 0	01010 " 12 " $t4	00111 " 7 " $a3	01101 " $ 13 " $t5	00000 " 0	100010 " 34 " Sub

∴ 0x01854020 = Add $t0, $t4, a1

∴ 0x02264824 = and $t1, $S1, $a2

∴ 0x01476822 = Sub $t5, $t4, $a3

5) Convert to Machine instruction (Assembly) from Hexadecimal Code.

Given,

0x02525020, 0x01646024, 0x01a37825

Hex Code	Binary	OPCode	rs	rt	rd	shamt	funct	Assembly
0x02525020	000000 10010 10010 01010 00000 100000	000000 " 0	10010 " 18 " $s2	10010 " 18 " $s2	01010 " 12 " $t4	00000 " 0	100000 " 32 " Add	Add $t4, $s2, $s2 Ans
0x01646024	000000 01011 00100 01100 00000 100100	000000 " 0	01011 " 11 " $t3	00100 " 4 " $a0	01100 " 12 " $t4	00000 " 0	100100 " 36 " and	and $t4, $t3, $a0 Ans
0x01A37825	000000 01101 00011 01111 00000 100101	000000 " 0	01101 " 13 " $t5	00011 " 3 " $v1	01111 " 15 " $t7	00000 " 0	100101 " 37 " or	or $t7, $t5, $v0 Ans

6) Assembly to machine code:

add = $s0, $t2, $t3

sub = $s1, $t5, $t6

and = $s2, $t8, $t9

or = $s3, $s4, $s5

sll = $t4, $s6, 3

MIPS ASSEMBLY	opcode	rs	rt	rd	Shamt	funct	Binary	Hexadecimal
add $s0, $t2, $t3	000000	(01010) 10 = t2	(01011) 11 = t3	(10000) 16 = $s0	00000 0	100000 32 = add	000000 01010 01011 10000 00000 100000	0x14B8020
Sub $s1, $t5, $t6	000000	01101 13 = $t5	01110 14 = $t6	10001 17 = $s1	00000 0	100010 34 = Sub	000000 01101 01110 10001 00000 100010	0x1AE8822
and $s2, $t8, $t9	000000	11000 = 24 = $t8	11001 = 25 = $t9	10010 18 = $s2	00000 0	100100 36 = and	000000 11000 11001 10010 00000 100100	0x03199024
or $s3, $s4, $s5	000000	10100 20 = $s4	10101 21 = $t5	10011 19 = $s3	00000 0	100101 37 = or	000000 10100 10101 10011 00000 100101	0x02959825
sll	000000 0	00000 0 Shifting Value Put here	10110 22 = $s6	01100 12 = $t4	00011 shift ammount = 3	000000 0		0x001660C0

7)

Instruction	Binary Group by field, op, rs, rt, rd, shat, funct	Hex
Sub $t0, $s1, $s2	000000, 10001, 10010, 01000, 00000, 100010	0x2324022
add $t1, $s3, $s4	000000, 10011, 10100, 01001, 00000, 100000	0x02744820
or $t2, $s5, $s6	000000, 10101, 10110, 01010, 00000, 100101	0x02B65025
and $t3, $s7, $t0	000000, 10111, 01000, 01011, 00000, 100100	0x02E85824
sll $t4, $s6, 3	000000, 00000, 01001, 01101, 00100, 000010	0x00096902

8) Convert a Code C to MIPS

Given, $x = ((y+z) - (z+w)) << 2;$

Let,

$x = \$S0$ $\quad \$t0 = (y+z)$

$y = \$S1$ $\quad \$t1 = (z+w)$

$z = \$S2$ $\quad \$t2 = (\$t0 - \$t1)$

$w = \$S3$

Step	operation	MIPS Assembly	Binary	Hexadecimal
1	y+z	Add $t0, $s1, $s2	000000, 10001, 10010, 01000, 00000, 100000	0x02324020
2	z+w	Add $t1, $s2, $s3	000000, 10010, 10011, 01001, 00000, 100000	0x02534820
3	(y+z)-(z+w)	Sub $t2, $t0, $t1	000000, 01000, 01001, 01010, 00000, 100010	0x01095022
4	2bit left shift	sll $S0, $t2, 2 -> Not used in directly	000000, 00000, 01010, 10000, 00010, 000000	0x000A8080

9) Convert Code C to MIPS:

Given, $P = ((a-b) + (b+c)) >>> 1$

Let, $P = \$S0$ $\quad \$t0 = a-b$

$a = \$S1$ $\quad \$t1 = b+c$

$b = \$S2$ $\quad \$t2 = (a-b)+(b+c)$

$c = \$S3$ $\quad P = t2 >>> 1$

Step	operation	MIPS Assembly	Binary	Hexadecimal
1	a-b	Sub $t0, $s1, $s2	000000, 10001, 10010 01000, 00000, 100010	0x02324022
2	b+c	Add, $t1, $s2, $s3	000000, 10010, 10011, 01001, 00000, 100000	0x02534820
	(a-b)+(b+c)	Add $t2, $t0, $t1	000000, 01000, 01001, 01010, 00000, 100000	0x01095020
	P = $t2 >>> 1	Srl $S0, $t2, 1 ↳2	000000, 00000, 01010, 10000, 00001, 000010	0x000A8042

10) what are the Primary Addressing modes supported in MIPS ?

Ans : Addressing modes specify how an instruction accesses its operands or determines the location of data.

The Primary Addressing modes in MIPS are :

Addressing Mode	Explaination	Example
① Register Addressing	• operands are stored in CPU Registers	add $t0, $t1, $t
② Immediate Addressing	• A constant value is provided directly in the instruction	addi $t0, $t1, 2
③ Base/Displacement Addressing	• The memory address is calculated by adding and offset to a base Register	lw $t0, 4($s0)
④ PC-Relative Addressing	• The target address is calculated relative to the Program Counter (PC). It is commonly used in branch instructions.	~~beq $t0, $t1,~~ beq, $t0, $t1, label
⑤ Pseudo-Direct Addressing	• A jump target is formed using part of the PC and the instructions address field	j target

* Fasted addressing mode : Register Addressing is generally considered the fasted because the operands are already available in CPU Registers. it avoid the additional memory access required to load data from main memory.

11) Different instruction format in MIPS Assembly.

Ans MIPS Architecture supports three primary instruction format. R-format, I-format, J-format. Each Instructio is 32 bits long, but their field arrangements and purposes are different

Particulars	R-format (Register)	I-format (Immediate)	J-format (Jump)
Definition	used for arithmetic and logical operation on register	used for immidiate values, memory access, and conditional branching	used for Jump instru-ction to transfer Program Execution
Structure	opcode | rs | rt | rd | shamt | funct	opcode | rs | rt | immidiate	opcode | target address
Field size	6+5+5+5+5+6 = 32 Bits	6+5+5+16 = 32 Bits	6+26 = 32 Bits
Field & functions	opcode (6) = Identifies the instruction type. rs (5) = First source Register. rt (5) = Target Register rd (5) = Destination Regist Shamt (5) = Shift ammount funct (6) = specifies the operation	opcode (6) = Identifies the instruction. rs (5) = Source or Base Register. rt (5) = Target Register Immidiate (16) = Contain a Constant Value or offset	opcode (6) = Identifies the Jump instruction Target Address (26): Contains the Encoded Jump target.
Example	add $t0, $t1, $t2	addi $t0, $t1, 10	j target
Explanation	Adds the values in $t1 and $t2 and Store the result in $t0	Add 10 to the $t1 and stores the result in $t0	Transfer Program Execution to the specified Jump target.

Conclusion: R-format is mainly used for Register operations, I format for immidiate, memory, and branch operations and J-format for jump operation. All three formats are 32 bits long but their field arrangements Different.